

Bilevel Fishery — debug & performance handoff

What changed on `fix/bilevel-fishery-debug`, why, and how to apply it

Prepared for Nadine · branch `fix/bilevel-fishery-debug`

2026-07-22

Contents

Executive summary	1
1. Measured impact (the two headline results)	3
Per-iteration wall time — the quadratic slowdown is gone	3
Memory per generation — the Algorithm-rebuild leak is gone	3
The outer optimiser now actually converges	3
2. What was broken and how it was fixed	5
2.1 Startup crash — Ray <code>local_mode</code>	5
2.2 The whole-machine freeze — <code>setproctitle</code> → <code>launchservicesd</code>	5
2.3 Thread oversubscription	6
2.4 Ray uploaded the repo + <code>.venv</code> on every run	6
2.5 Outer ES was degenerate (population = 1) — correctness	6
2.6 <code>sigma</code> never annealed — correctness	6
2.7 W&B <code>es/search_mean</code> frozen / off-by-one	6
2.8 Quadratic slowdown A — Algorithm rebuilt every generation	6
2.9 Quadratic slowdown B — <code>env_reduced</code> re-plotted a growing table	6
3. Decisions that are yours to make	8
4. How to apply this on your side	9

Executive summary

The goal was to run `examples/bilevel_fishery/debug.py` end-to-end on a standard macOS laptop and fix whatever prevented it. Three classes of problem were found and fixed; all fixes are **measured**, minimal, and backward-compatible.

#	Problem	Status
1	Crash at startup (Ray <code>local_mode</code> + working-dir capture)	Fixed
2	Whole machine froze during training (Ray <code>setproctitle</code> → macOS <code>launchservicesd</code>)	Fixed
3	Thread oversubscription (~1000 threads / 16 cores)	Fixed
4	Ray silently uploaded the repo + 1.2 GB <code>.venv</code> on every run	Fixed
5	Outer ES was degenerate (population = 1 → zero gradient signal)	Fixed

#	Problem	Status
6	ES sigma never annealed (search could not converge)	Fixed
7	W&B es/search_mean plots frozen / off-by-one	Fixed
8	Quadratic slowdown: RLlib Algorithm rebuilt every generation (memory leak)	Fixed
9	Quadratic slowdown: env_reduced re-plotted a growing table every iteration	Fixed

Net effect: the experiment now **starts, runs without freezing the machine, keeps the CPU/RAM bounded, has a working (non-degenerate, annealing) outer optimiser, and runs in linear time** (~6 min/generation, constant) instead of quadratic.

Two items (#5, #6) are **correctness** bugs, not just performance: in the shipped debug.py configuration the outer optimiser was not actually optimising. Any result produced in that configuration before these fixes should be treated with caution.

Nothing has been pushed to your branch. Everything lives on the parallel branch `fix/bilevel-fishery-debug`. See *How to apply* at the end.

1. Measured impact (the two headline results)

Per-iteration wall time — the quadratic slowdown is gone

Before the fixes, each inner iteration got steadily slower ($\approx 37 \text{ s} \rightarrow \approx 190 \text{ s}$ over the first 4 generations, still climbing) — total runtime was **quadratic**, so 1000 generations was unreachable. After the fixes, per-iteration time is **flat** ($\sim 3 \text{ s}$) across 2600+ iterations, with a single $\sim 15\text{--}40 \text{ s}$ plotting iteration once per generation.

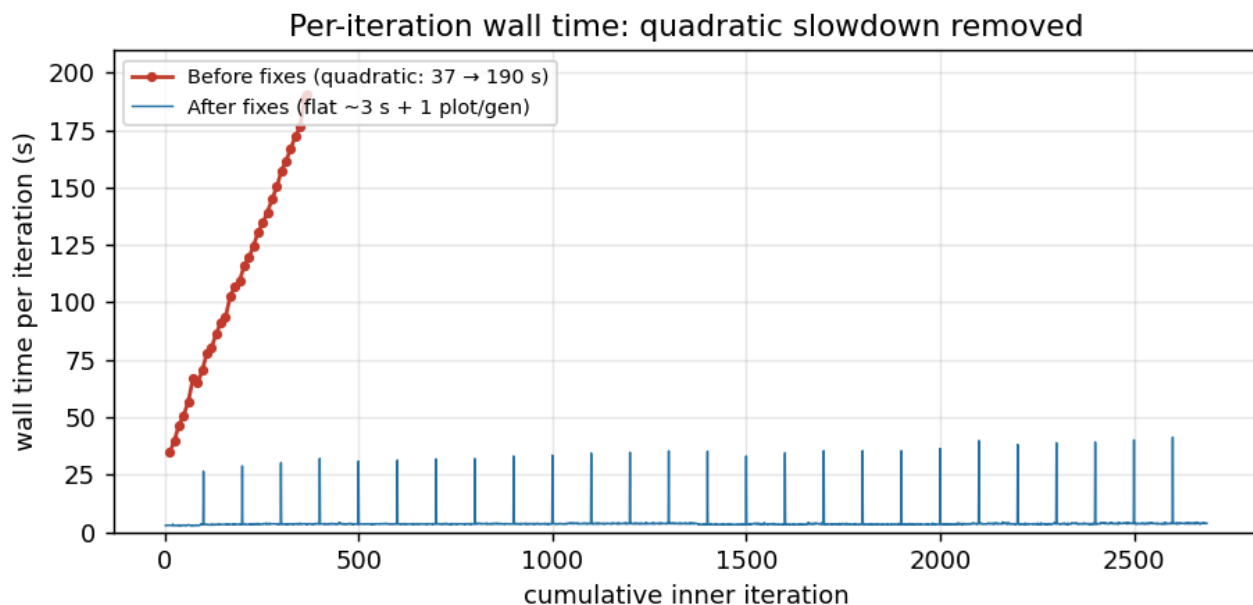


Figure 1: Per-iteration wall time, before vs after. Same workload (1600 env-steps/iter); the growth was pure overhead, now removed.

Memory per generation — the Algorithm-rebuild leak is gone

`PolicyActor.reset()` rebuilt a whole RLlib Algorithm every generation and never freed the old one ($\sim 20 \text{ MB/gen}$, unbounded). Calling `stop()` first does **not** reclaim it. Resetting the weights in place bounds the memory (plateau).

The outer optimiser now actually converges

Fixes #5 (population) and #6 (sigma annealing) turned a degenerate outer search into a working one: the best mechanism improves, then the search spread sigma contracts monotonically over generations (live run below).

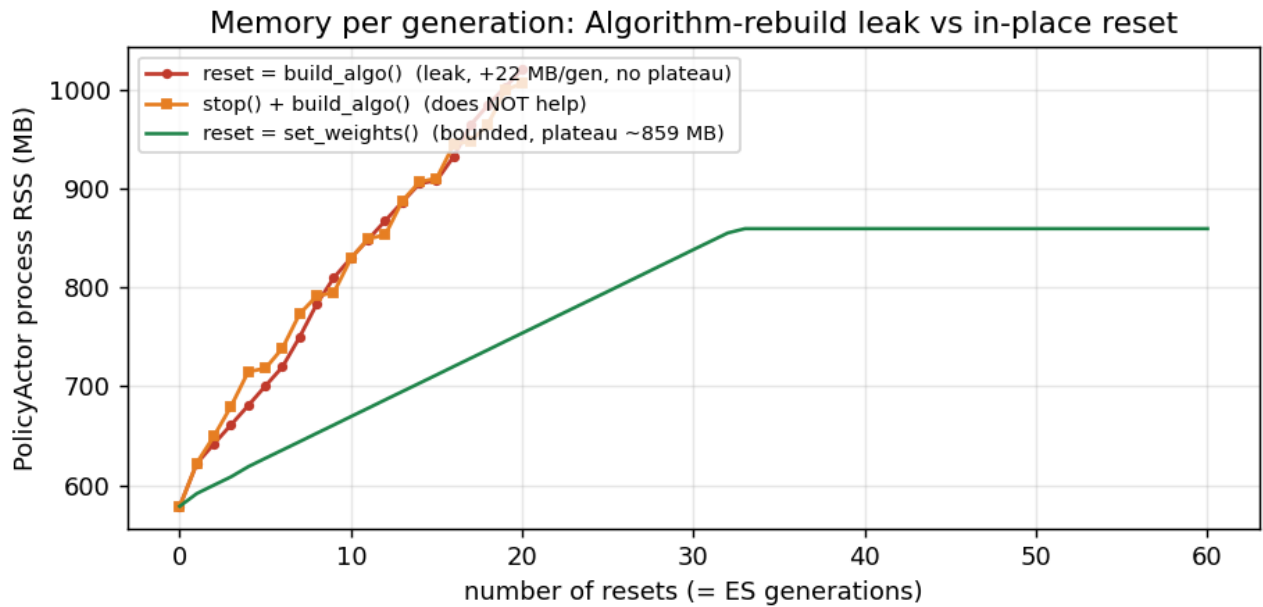


Figure 2: PolicyActor process RSS across resets. Rebuild leaks unboundedly; stop() +rebuild does not help; in-place set_weights plateaus.

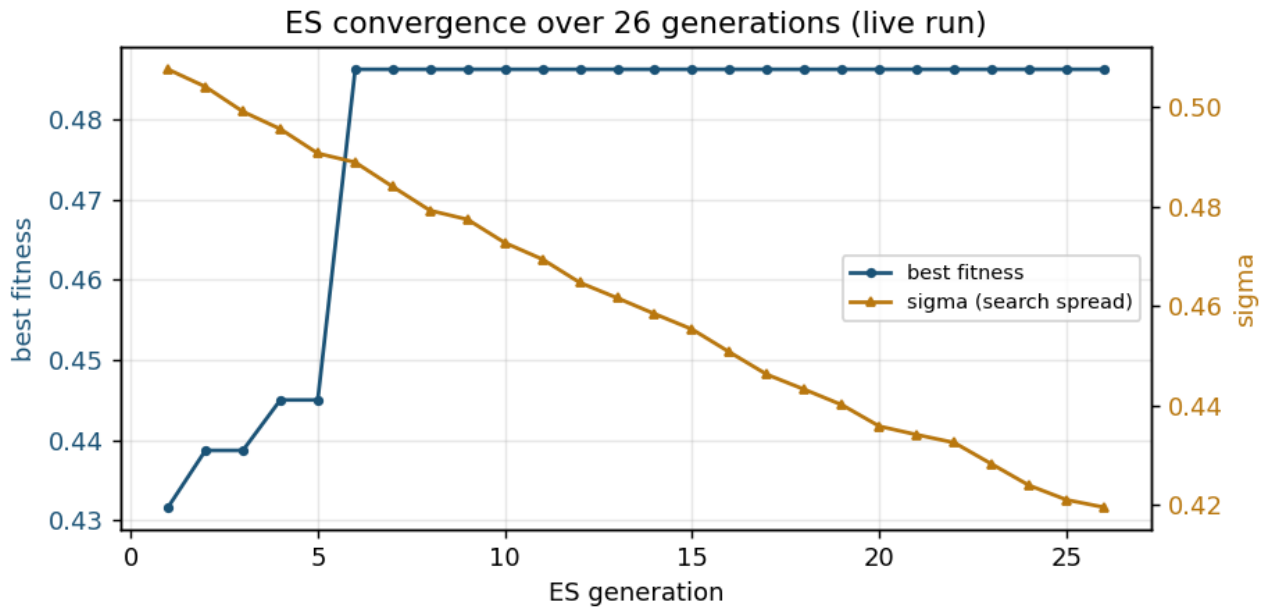


Figure 3: ES convergence: best fitness rises to a plateau while sigma anneals — the search is genuinely exploring and contracting, not stuck.

2. What was broken and how it was fixed

Grouped by theme. File references are on `fix/bilevel-fishery-debug`.

2.1 Startup crash — Ray `local_mode`

`core/adaptors/ray/runtime.py` hard-coded `ray.init(local_mode=True)`. On Ray 2.53, launching from inside the editable repo makes Ray capture the repo as `working_dir`, which local mode cannot upload → `ValueError: ... is not a valid URI`.

Fix. `RayRuntimeConfig` gained `local_mode` / `log_to_driver` fields (default `False`); `BilevelConfig.ray()` passes them through. Use `local_mode=True` only for step-debugging, and launch from outside the repo. (*commit 3f1a324; runtime.py, bilevel.py, debug.py*)

2.2 The whole-machine freeze — `setproctitle` → `launchservicesd`

The hard one. Ray renames every worker (`ray::PolicyActor.train, ...`) **before and after each task**. On macOS that rename is a *synchronous* XPC round-trip to the root daemon `launchservicesd`. With ~19 workers renaming thousands of times, that single daemon's serial queue saturates and holds a system-wide lock — the UI freezes **while the CPU is 50–90 % idle** (macOS load average is meaningless here). This is why closing apps and rebooting didn't help.

Confirmed by sampling a live worker (`sample <pid>`, no `sudo`): the stack went `execute_task` → `_changeproctitle` → `ray._raylet.setproctitle` → `darwin_set_process_title` → `_LSSetApplicationInformationItem` → `xpc_connection_send_message_with_reply_sync`.

Fix. No-op the per-task rename (it is cosmetic — only the Activity-Monitor label): a `worker_process_setup_hook` plus a driver-side patch. Measured: `launchservicesd` 190–214 % → ~0 %. (*commits ae4dc72, 194a299; new file core/adaptors/ray/_worker_hooks.py, runtime.py*)

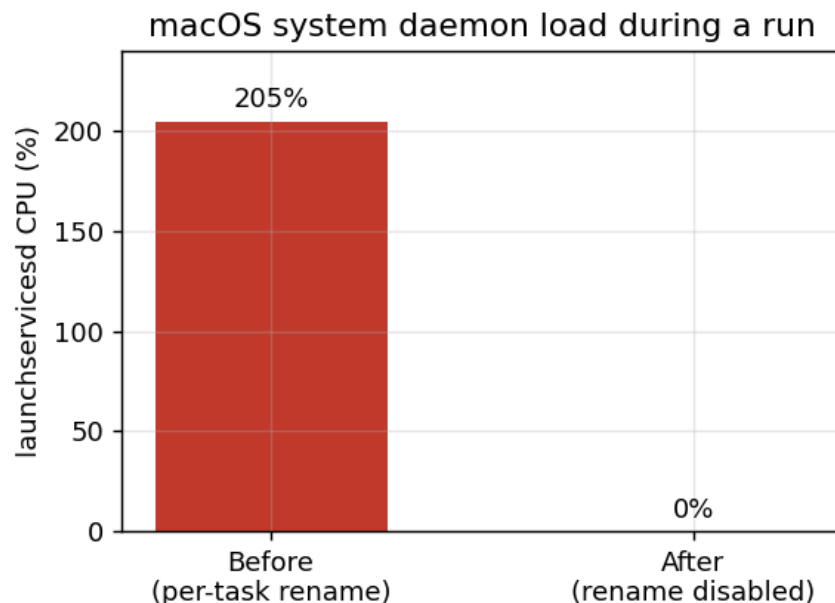


Figure 4: macOS `launchservicesd` CPU during a run, before vs after disabling the per-task rename.

2.3 Thread oversubscription

Measured ~1083 threads across 31 Ray processes on 16 cores; the run-queue saturated and starved WindowServer. Root cause: NumPy links Apple Accelerate, which **ignores** `OMP_NUM_THREADS` (needs `VECLIB_MAXIMUM_THREADS`), and Torch defaults to 12 intra-op threads.

Fix. `RayRuntimeConfig` now caps `OMP/OPENBLAS/MKL/VECLIB/NUMEXPR` on the driver **and** propagates them to every worker via `runtime_env["env_vars"]` (so they apply before NumPy/Torch import), plus `torch.set_num_threads(1)`. `omp_threads` is the tuning knob. (*commit 01ff4b0; runtime.py*)

2.4 Ray uploaded the repo + .venv on every run

Launching with `uv run` made Ray capture the whole cwd (incl. the 1.2 GB `.venv`) as `working_dir` into `/tmp/ray` — every run. **Fix.** Launch with `.venv/bin/python + PYTHONPATH`, and inject `PYTHONPATH` for workers instead of uploading. `/tmp/ray` now stays at a few hundred KB. (*commit d04d860; runtime.py*)

2.5 Outer ES was degenerate (population = 1) — correctness

`batch_capacity = num_envs_per_env_runner // len(inner_seeds)`. With `num_envs_per_env_runner = 1`, population was 1: `mean = best = worst, variance = 0, gradient ≈ 0`. The outer search could not move.

Fix. `num_envs_per_env_runner = 16` in `debug.py` (→ population 16, 8 antithetic pairs). A second hard-coded `population == 1` assumption in the reporting util was also relaxed. Verified: candidates now get genuinely different fitness (`var > 0`). (*commits 8fd9942, 4deeb00; debug.py, es/optimizer.py*)

2.6 sigma never annealed — correctness

The 1/5 success rule was computed on **whitened** fitness (zero-mean by construction → “success” ≈ 0.5 every generation → `sigma` pinned), plus a soft-anchor pulled `sigma` back up. The distribution could never contract.

Fix. Success rate is now computed on **raw** fitness vs the previous generation’s best incumbent, and the soft-anchor is removed. Near an optimum the success rate drops below the 0.2 target, so `sigma` anneals down and the search converges. Verified live: `sigma` 0.508 → 0.42 monotonically (Fig. 3). (*commit 5823d6e; es/optimizer.py*)

2.7 W&B es/search_mean frozen / off-by-one

The reporter logged pre-update `mean/sigma` and a `generation+1` label, so the search-centre curves looked stuck at the initial 0.5. **Fix.** Report the current (post-update) centre and the correct generation index. (*commit 5823d6e; es/optimizer.py*)

2.8 Quadratic slowdown A — Algorithm rebuilt every generation

`PolicyActor.reset()` did `self.algo = self.algo_config.build_algo()` every generation. RLlib repopulates global registries / connector state on each build that are **not** freed by garbage collection (nor by `stop()` — verified), leaking ~20 MB/gen (Fig. 2). **Fix.** Reset the policy in place with `self.algo.set_weights(self._init_weights)` (the method’s original, commented-out intent): RSS plateaus, reset is ~100× cheaper. **Decision for you** — see §3. (*commit 5e219c9; policy_actor.py*)

2.9 Quadratic slowdown B — env_reduced re-plotted a growing table

`plot_env_reduced` (called every inner iteration) appends to a module-global per-metric wandb. Table that is never reset, then re-renders a plot from the **entire** accumulated history each iteration → $O(\text{iterations}^2)$. **Fix.** Throttle the plot to **once per generation** (last inner iteration, i.e. the trained policy) via a `_train_iters_hint`

the regulator passes to the inner optimiser; per-iteration behaviour is preserved when the hint is unset. (*commit 5e219c9; optimizer.py, regulator.py*)

3. Decisions that are yours to make

These fixes work, but three points are **methodological / design choices** that we deliberately did not decide for you:

1. **set_weights vs full rebuild (§2.8).** `set_weights` restores the initial `RLModule` weights but does **not** reset the learner optimiser state (Adam moments), RNG, or connector running statistics. If your intended semantics is a *fully fresh* optimiser per generation, a leak-free rebuild path is needed instead (e.g. a short-lived `PolicyActor` per generation that is `stop()`-ed and killed). If “reset the weights” is enough, the current fix is ideal.
2. **env_reduced cadence (§2.9).** The trajectory plot now has **per-generation** resolution instead of per-inner-iteration. `_train_iters_hint` is the single knob. Alternatively, make `env_reducers` fully opt-in (do not default to `build_default_fishery_reduction_specs()` in `RayOptimizer.__init__`).
3. **Inner-loop equilibrium validity (scientific).** The inner RL policy is reset each generation and, in the observed runs, its returns are **still rising** at the end of the generation’s 100 iterations — i.e. it may be scored by the ES before reaching equilibrium. For a bilevel/Stackelberg formulation the follower equilibrium must be (approximately) reached for the leader’s fitness to be meaningful. Worth checking (more `train_iters`, warm-start, or a convergence criterion). This is the point an ML reviewer will probe first.

Known remaining issue (minor, not fixed): the `World` actor’s `_opt_ctx_map[opt_id]` is append-only (never trimmed by `flush_ctx`), growing ~6400 ids/iteration. This is a **memory** leak only (CPU cost negligible; the reducer cursor keeps per-call work $O(\text{new})$). A safe fix is to drop the already-consumed prefix in `flush_ctx` and rebase the cursor. Left for you because it touches the reducer cursor semantics.

Other pre-existing quirks (unchanged): `outer.debugging(num_seeds=10)` is dead (`bilevel.py` overrides `outer` seeds with the inner’s single seed); `policy_loss` is reported as NA (not extracted from APPO results).

4. How to apply this on your side

All changes are on **fix/bilevel-fishery-debug** (branched from a common ancestor of feat/fresh-water-rework). 10 commits, 8 code files (+208 / -33 lines), plus docs.

Commits (newest first):

```
5e219c9 fix: remove quadratic slowdown (algo rebuild + env_reduced replot)
5823d6e fix(es): anneal sigma (raw-fitness 1/5 rule) + fix wandb es reporting
194a299 docs: record setproctitle/launchservicesd freeze root cause + fix
ae4dc72 fix(ray): disable per-task setproctitle on macOS (freeze)
f50a422 docs: RUNBOOK + TOREAD handoff notes
d04d860 fix(ray): stop uploading the repo/.venv to workers
8fd9942 feat: set ES population to 16 (num_envs_per_env_runner)
4deeb00 fix(es): let ES reporting support population > 1
01ff4b0 fix(ray): cap BLAS/torch threads and propagate to workers
3f1a324 fix(ray): make local_mode/log_to_driver configurable (default False)
```

(8a865e9, your double-training fix, is already shared and not re-applied.)

Recommended review flow. Look at the branch, then either merge it wholesale or cherry-pick the commits you want:

```
git fetch origin
git log --oneline origin/feat/fresh-water-rework..origin/fix/bilevel-fishery-debug
git diff origin/feat/fresh-water-rework..origin/fix/bilevel-fishery-debug -- '*.py'
```

Option A – take everything:

```
git switch feat/fresh-water-rework
git merge origin/fix/bilevel-fishery-debug
```

Option B – take specific fixes (example: the perf + ES ones):

```
git cherry-pick 3f1a324 01ff4b0 d04d860 4deeb00 8fd9942 ae4dc72 5823d6e 5e219c9
```

How to run it (note: **not** `uv run` — that is what uploaded the 1.2 GB `.venv`):

```
WANDB_MODE=disabled PYTHONPATH=. .venv/bin/python examples/bilevel_fishery/debug.py
```

Remove `WANDB_MODE=disabled` (after `wandb login`) to log to W&B as before.

Companion documents on the branch: `examples/bilevel_fishery/RUNBOOK.md` (technical), `TOREAD.md` (plain-language), `PERF_FINDINGS.md` (the full performance diagnosis with measurements).